

Lecture 12 – Garbage Collection

COSE212: Programming Languages

Jihyeok Park



2026 Fall

- Mutation makes it possible to **change the state** of a program by **updating the contents** of a data structure or a variable.
 - BFAE – FAE with **mutable boxes**
 - MFAE – FAE with **mutable variables**
 - Evaluation with **memories**, finite maps from addresses to values:

$$\begin{array}{ll} \text{Memories} & M \in \mathbb{A} \xrightarrow{\text{fin}} \mathbb{V} \\ \text{Addresses} & a \in \mathbb{A} \end{array}$$

- In this lecture, we will learn **memory management** techniques to **deallocate** unreachable memory cells:
 - **Stack and Heap**
 - **Manual Memory Management**
 - **Garbage Collection (GC)**

1. Stack and Heap

Tail-Call Optimization (TCO)

2. Manual Memory Management

3. Garbage Collection

Reference Counting

Mark-and-Sweep GC

Copying GC (Two-Space GC)

Other Techniques

1. Stack and Heap

Tail-Call Optimization (TCO)

2. Manual Memory Management

3. Garbage Collection

Reference Counting

Mark-and-Sweep GC

Copying GC (Two-Space GC)

Other Techniques

In the previous lecture, we saw that the memory of the following MFAE expression has **unreachable** memory cells:

```
/* MFAE */
var y = 1;
var f = x => {
  x = x + y;
  x * x
};
f(5);           /* 36 */
y = 3;
f(5)            /* 64 */
```

$$\sigma = [$$

$$y \mapsto a_0$$

$$f \mapsto a_1$$

$$]$$

$$\mathbb{A} : a_0 \quad a_1 \quad a_2 \quad a_3 \quad \dots$$

$$M = \begin{array}{|c|c|c|c|c|} \hline 3 & v & 6 & 8 & \dots \\ \hline \end{array}$$

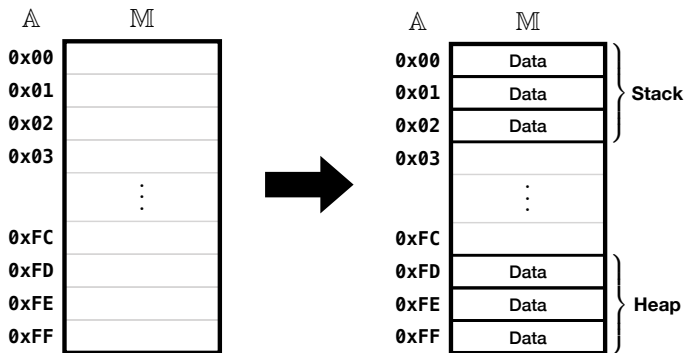
where $v = \langle \lambda x. (x = x + y; x * x), [y \mapsto a_0] \rangle$

Then, how can we **detect** and **deallocate** unreachable memory cells?

Let's delete **unreachable** memory cells when the program **exits** functions!

We can **divide** the memory into two parts:

- **Stack** – for **local variables** and **function parameters**
- **Heap** – for **dynamically allocated** memory cells



Create a **new stack frame** when the program **enters** a function, and **delete** the stack frame when it **exits** the function.

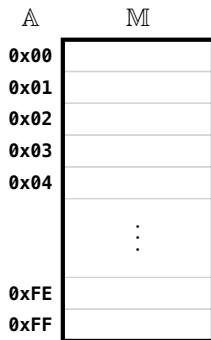
For example, consider the following Scala program:

```
case class Box(var k: Int)

def f(x: Int): Int =
  var y = Box(1)
  var z = g(2)
  x + y.k + z

def g(b: Int): Int =
  var c = Box(b)
  c.k + 3

var a = 1
var b = f(42)
a + b
```



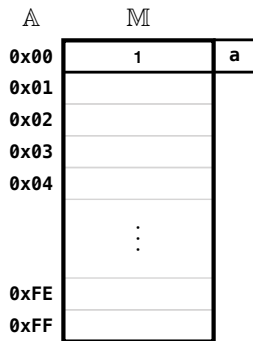
For example, consider the following Scala program:

```
case class Box(var k: Int)

def f(x: Int): Int =
  var y = Box(1)
  var z = g(2)
  x + y.k + z

def g(b: Int): Int =
  var c = Box(b)
  c.k + 3

var a = 1                /* a -> 0x00 */
var b = f(42)
a + b
```



For example, consider the following Scala program:

```
case class Box(var k: Int)

def f(x: Int): Int = /* x -> 0x01 */
  var y = Box(1)      /* y -> 0x02 */ *
  var z = g(2)
  x + y.k + z

def g(b: Int): Int =
  var c = Box(b)
  c.k + 3

var a = 1              /* a -> 0x00 */
var b = f(42)
a + b
```

A	M	
0x00	1	a
0x01	42	x
0x02	0xFF	y
0x03		
0x04		
	⋮	
0xFE		
0xFF	1	

f

A **new stack frame** is created when the program enters the function f.

For example, consider the following Scala program:

```
case class Box(var k: Int)

def f(x: Int): Int = /* x -> 0x01 */
  var y = Box(1)      /* y -> 0x02 */
  var z = g(2)
  x + y.k + z

def g(b: Int): Int = /* b -> 0x03 */
  var c = Box(b)      /* c -> 0x04 */
  c.k + 3              /* 5 */

var a = 1              /* a -> 0x00 */
var b = f(42)
a + b
```

A	M	
0x00	1	a
0x01	42	x
0x02	0xFF	y
0x03	2	b
0x04	0xFE	c
	:	
	:	
0xFE	2	
0xFF	1	

A **new stack frame** is created when the program enters the function `g`.

For example, consider the following Scala program:

```
case class Box(var k: Int)

def f(x: Int): Int = /* x -> 0x01 */
  var y = Box(1)      /* y -> 0x02 */
  var z = g(2)        /* z -> 0x03 */
  x + y.k + z         /* 48 */ *

def g(b: Int): Int =
  var c = Box(b)
  c.k + 3

var a = 1              /* a -> 0x00 */
var b = f(42)
a + b
```

A	M	
0x00	1	a
0x01	42	x
0x02	0xFF	y
0x03	5	z
0x04	⋮	
0xFE	2	
0xFF	1	

After exiting the function `g`, its stack frame is **deleted**. The memory cells allocated for `b` and `c` in the stack frame are **deallocated**.

For example, consider the following Scala program:

```
case class Box(var k: Int)

def f(x: Int): Int =
  var y = Box(1)
  var z = g(2)
  x + y.k + z

def g(b: Int): Int =
  var c = Box(b)
  c.k + 3

var a = 1           /* a -> 0x00 */
var b = f(42)       /* b -> 0x01 */
a + b               /* 49 */
```

A	M	
0x00	1	a
0x01	48	b
0x02		
0x03		
0x04		
	⋮	
0xFE	2	
0xFF	1	

After exiting the function `f`, its stack frame is **deleted**. The memory cells allocated for `x`, `y`, and `z` in the stack frame are **deallocated**.

Tail-Call Optimization (TCO)

Here is another example with a recursive function:

```
def sum(x: Int, acc: Int): Int =  
  if (x < 1) acc  
  else sum(x - 1, x + acc)  
  
sum(1000, 0)
```

A	M		
0x00	1000	x	sum
0x01	0	acc	
0x02			
0x03			
0x04			
0x05			
	⋮		
0xFE			
0xFF			

Tail-Call Optimization (TCO)

Here is another example with a recursive function:

```
def sum(x: Int, acc: Int): Int =  
  if (x < 1) acc  
  else sum(x - 1, x + acc)  
  
sum(1000, 0)
```

A	M		
0x00	1000	x	sum
0x01	0	acc	
0x02	999	x	sum
0x03	1000	acc	
0x04			
0x05			
	⋮		
0xFE			
0xFF			

Tail-Call Optimization (TCO)

Here is another example with a recursive function:

```
def sum(x: Int, acc: Int): Int =  
  if (x < 1) acc  
  else sum(x - 1, x + acc)  
  
sum(1000, 0)
```

A	M		
0x00	1000	x	sum
0x01	0	acc	
0x02	999	x	sum
0x03	1000	acc	
0x04	998	x	sum
0x05	1999	acc	
	:	:	
0xFE	873	x	sum
0xFF	118999	acc	

It fails with a **stack overflow** error.

However, is it really necessary to keep **all the stack frames**? **No!**

Scala supports **tail-call optimization** (TCO).

Tail-Call Optimization (TCO)

Here is another example with a recursive function:

```
def sum(x: Int, acc: Int): Int =  
  if (x < 1) acc  
  else sum(x - 1, x + acc) // tail-call  
  
sum(1000, 0)
```

A	M		
0x00	1000	x	sum
0x01	0	acc	
0x02			
0x03			
0x04			
0x05			
	⋮		
0xFE			
0xFF			

Why? The function call is in the **tail-call position** (i.e., the final action in the function). It means that it directly returns the result without any further computation.

Thus, we can safely **discard** the current stack frame **before** calling the function, and it is called **tail-call optimization** (TCO).

Tail-Call Optimization (TCO)

Here is another example with a recursive function:

```
def sum(x: Int, acc: Int): Int =  
  if (x < 1) acc  
  else sum(x - 1, x + acc) // tail-call  
  
sum(1000, 0)
```

A	M		
0x00	999	x	sum
0x01	1000	acc	
0x02			
0x03			
0x04			
0x05			
	⋮		
0xFE			
0xFF			

Why? The function call is in the **tail-call position** (i.e., the final action in the function). It means that it directly returns the result without any further computation.

Thus, we can safely **discard** the current stack frame **before** calling the function, and it is called **tail-call optimization** (TCO).

Tail-Call Optimization (TCO)

Here is another example with a recursive function:

```
def sum(x: Int, acc: Int): Int =  
  if (x < 1) acc  
  else sum(x - 1, x + acc) // tail-call  
  
sum(1000, 0) // 500500
```

A	M		
0x00	0	x	sum
0x01	500500	acc	
0x02			
0x03			
0x04			
0x05			
	⋮		
0xFE			
0xFF			

Why? The function call is in the **tail-call position** (i.e., the final action in the function). It means that it directly returns the result without any further computation.

Thus, we can safely **discard** the current stack frame **before** calling the function, and it is called **tail-call optimization** (TCO).

```
def factorial(x: Int): Int =  
  if (x < 2) 1  
  else factorial(x - 1) * x
```

Is it in the **tail-call position**? **No!**

After `factorial(x - 1)`, it needs to multiply the result by `x`.

```
def factorial(x: Int): Int =  
  if (x < 2) 1  
  else x * factorial(x - 1)
```

Is it in the **tail-call position**? **Still No!**

After `factorial(x - 1)`, it still needs to multiply the result by `x`.

Then, how can we put it in the **tail-call position**?

One common pattern for TCO is to use an **accumulator**:

```
def factorial(x: Int, acc: Int): Int =  
  if (x < 2) acc  
  else factorial(x - 1, x * acc)  
  
factorial(5, 1) // 120
```

However, it is not a user-friendly interface because we need to pass the initial value of the accumulator (e.g., 1) every time. We can define a nested function to hide the additional parameter:

```
def factorial(x: Int): Int =  
  def aux(x: Int, acc: Int): Int =  
    if (x < 2) acc  
    else aux(x - 1, x * acc)  
  aux(x, 1)  
  
factorial(5) // 120
```

Many programming languages support **tail-call optimization** (TCO) to avoid stack overflow errors.

In addition, Scala supports the `@tailrec` annotation to check at compile time whether a recursive call is in the tail-call position:

```
import scala.annotation.tailrec

// Passes the tail-call position check
@tailrec
def factorial(x: Int, acc: Int): Int =
  if (x < 2) acc
  else factorial(x - 1, x * acc)

// Compile-time error
@tailrec
def factorial(x: Int): Int =
  if (x < 2) 1
  else x * factorial(x - 1)
```

1. Stack and Heap

Tail-Call Optimization (TCO)

2. Manual Memory Management

3. Garbage Collection

Reference Counting

Mark-and-Sweep GC

Copying GC (Two-Space GC)

Other Techniques

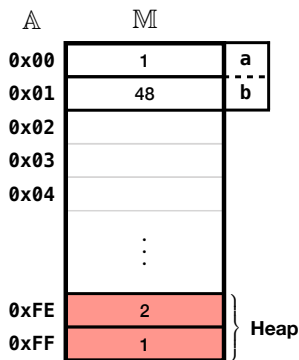
Let's see the previous example again:

```
case class Box(var k: Int)

def f(x: Int): Int =
  var y = Box(1)
  var z = g(2)
  x + y.k + z

def g(b: Int): Int =
  var c = Box(b)
  c.k + 3

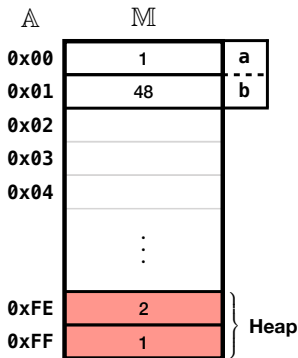
var a = 1           /* a -> 0x00 */
var b = f(42)       /* b -> 0x01 */
a + b               /* 49 */
```



Unfortunately, we still cannot deallocate memory cells (e.g., 0xFE and 0xFF) dynamically allocated in the **heap** rather than the **stack**.

One way to resolve this is **manual memory management**: e.g., C++ provides keywords for allocation (`new`) and deallocation (`delete`):

```
struct Box { int k; Box(int k): k(k) {} };
int f(int x) {
    Box* y = new Box(1);    // alloc  0xFF
    int z = g(2);
    int k = y->k;
    return x + k + z;
}
int g(int b) {
    Box* c = new Box(b);    // alloc  0xFE
    int k = c->k;
    return k + 3;
}
int a = 1;    /* a -> 0x00 */
int b = f(42); /* b -> 0x01 */
a + b;        /* 49 */
```



One way to resolve this is **manual memory management**: e.g., C++ provides keywords for allocation (`new`) and deallocation (`delete`):

```
struct Box { int k; Box(int k): k(k) {} };
int f(int x) {
    Box* y = new Box(1);    // alloc  0xFF
    int z = g(2);
    int k = y->k; delete y; // dealloc 0xFF
    return x + k + z;
}
int g(int b) {
    Box* c = new Box(b);    // alloc  0xFE
    int k = c->k; delete c; // dealloc 0xFE
    return k + 3;
}
int a = 1;    /* a -> 0x00 */
int b = f(42); /* b -> 0x01 */
a + b;        /* 49 */
```

A	M	
0x00	1	a
0x01	48	b
0x02		
0x03		
0x04		
	⋮	
0xFE		
0xFF		

Pros:

- **Efficient** – Users can **explicitly** deallocate memory cells allocated in heap whenever they want.

Cons:

- **Error-prone** – Users have all the **responsibility** to deallocate memory cells allocated in heap:
 - **Memory Leak** occurs if users forget to deallocate memory cells.

```
b = new Box(42); ...
```

- **Use After Free (UAF)** occurs if users try to access memory cells already deallocated.

```
b = new Box(42); ... delete b; ... b->k;
```

- **Double Free** occurs if users deallocate memory cells more than once.

```
b = new Box(42); ... delete b; ... delete b;
```

1. Stack and Heap

Tail-Call Optimization (TCO)

2. Manual Memory Management

3. Garbage Collection

Reference Counting

Mark-and-Sweep GC

Copying GC (Two-Space GC)

Other Techniques

Is there any way to **automatically** deallocate memory cells in heap? **Yes!**

Garbage collection (GC) is a representative technique for **automatic memory management** that automatically **detects** and **deallocates** memory cells that are no longer reachable in heap.

Let's learn several GC algorithms:

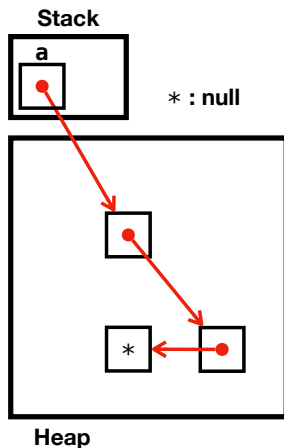
- **Reference counting**
- **Mark-and-sweep GC**
- **Copying GC** (Two-space GC)
- Others

Before explaining them, let's represent memory cells in heap in a **graphical way** without actual addresses.

From now on, we will use the **graphical representation** of memory cells:

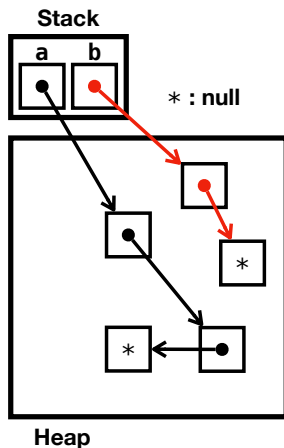
```
case class Box(var x: Box)
var a = Box(Box(Box(null))) *
var b = Box(Box(null))

a.x.x.x = a.x
b.x = a
a.x = Box(null)
a = a.x
b = null
```



From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)
var a = Box(Box(Box(null)))
var b = Box(Box(null)) *
a.x.x.x = a.x
b.x = a
a.x = Box(null)
a = a.x
b = null
```

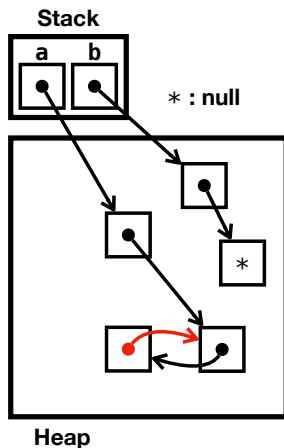


From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)
var a = Box(Box(Box(null)))
var b = Box(Box(null))

a.x.x.x = a.x
b.x = a
a.x = Box(null)
a = a.x
b = null
```

*



From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)
var a = Box(Box(Box(null)))
var b = Box(Box(null))

a.x.x.x = a.x

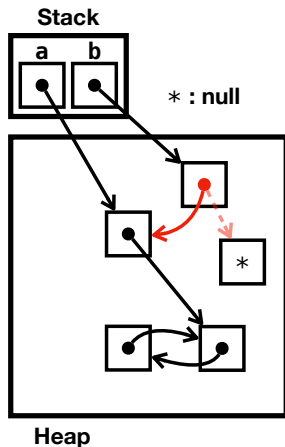
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*

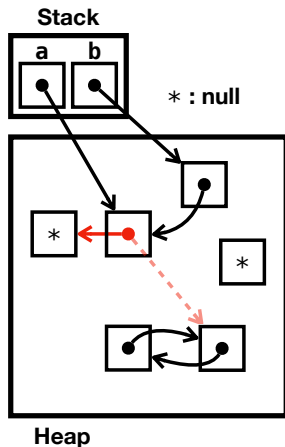


From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)
var a = Box(Box(Box(null)))
var b = Box(Box(null))

a.x.x.x = a.x
b.x = a
a.x = Box(null)
a = a.x
b = null
```

*



From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

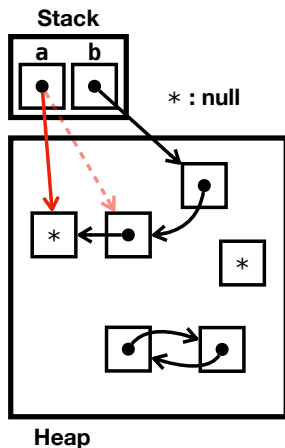
a.x.x.x = a.x

b.x = a

a.x = Box(null)

a = a.x

b = null
```

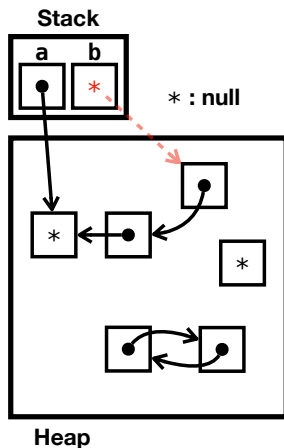


From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)
var a = Box(Box(Box(null)))
var b = Box(Box(null))

a.x.x.x = a.x
b.x = a
a.x = Box(null)
a = a.x
b = null
```

*

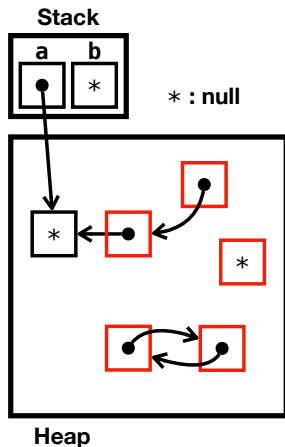


From now on, we will use the **graphical representation** of memory cells:

```
case class Box(var x: Box)
var a = Box(Box(Box(null)))
var b = Box(Box(null))

a.x.x.x = a.x
b.x = a
a.x = Box(null)
a = a.x
b = null
```

*



We need to deallocate five unreachable memory cells in heap.

Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

- 1 **Initialize** the reference count of each cell to 0.
- 2 When a reference to a cell is created, **increment** its reference count.
- 3 When a reference to a cell is deleted, **decrement** its reference count.
- 4 When the reference count of a cell reaches 0, **deallocate** the cell.

Many programming languages use reference counting to implement GC:

- Python, Swift, Perl, Objective-C, etc.

Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null))) *
var b = Box(Box(null))

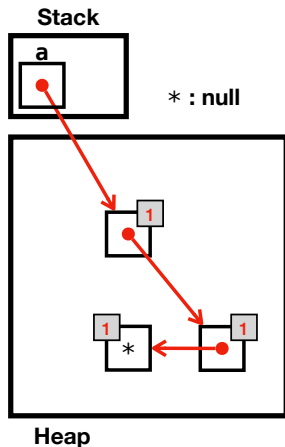
a.x.x.x = a.x

b.x = a

a.x = Box(null)

a = a.x

b = null
```



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null)) *

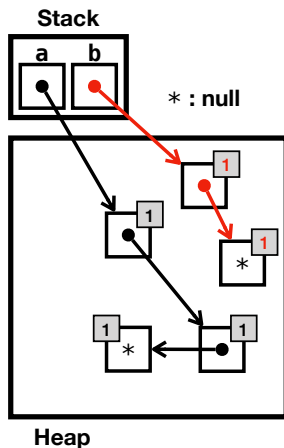
a.x.x.x = a.x

b.x = a

a.x = Box(null)

a = a.x

b = null
```



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

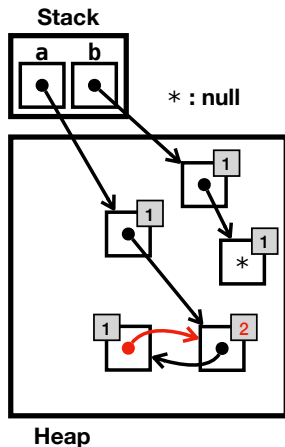
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

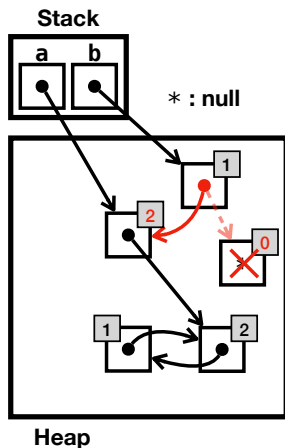
b.x = a

a.x = Box(null)

a = a.x

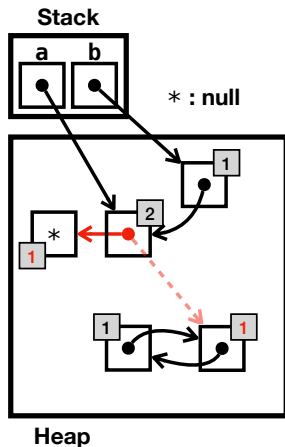
b = null
```

*



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

*



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

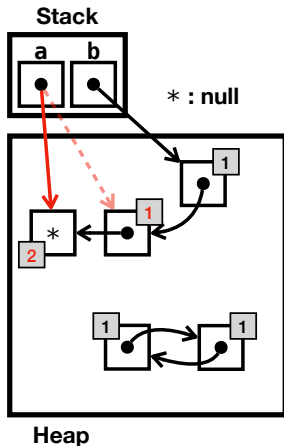
a.x.x.x = a.x

b.x = a

a.x = Box(null)

a = a.x

b = null
```



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

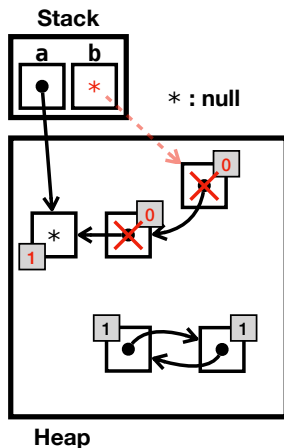
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

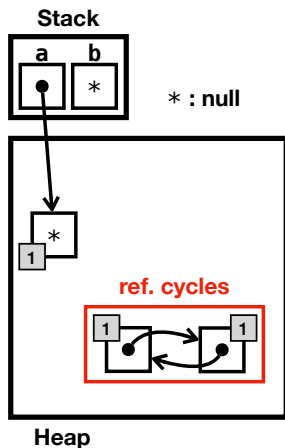
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Unfortunately, we cannot deallocate unreachable **reference cycles**.

Reference counting is a simple GC algorithm that keeps track of the **number of references** to each memory cell in heap.

Pros:

- **Easy to implement** – Simply increment and decrement the reference count when a reference is created and deleted.
- **No pauses** – Deallocation happens incrementally and immediately, without **stop-the-world** pauses.

Cons:

- **Reference cycles** – It cannot deallocate unreachable reference cycles.
- **Reference count cost** – It requires extra space and time to maintain reference counts.
- **Free List and Fragmentation** – It requires a **free list** to keep track of available free memory cells in heap, and it also suffers from **fragmentation** making it difficult to allocate large objects.

Mark-and-Sweep GC is one of the **tracing GC** algorithms that **traverses the heap** to find unreachable objects when it is **triggered** under some conditions.

- ① **Mark** all memory cells as **unreachable** (white).
- ② **Mark** all memory cells referenced by roots as **unscanned** (gray).
- ③ Repeat until there are no unscanned (gray) memory cells:
 - ① **Pick** an unscanned (gray) memory cell.
 - ② **Mark** memory cells referenced by the picked one as **unscanned** (gray).
 - ③ **Mark** the picked memory cell as **scanned** (black).
- ④ **Deallocate** (sweep) all memory cells that are still marked as **unreachable** (white).

Assume that the GC is triggered at the following program point, and let's perform **mark-and-sweep** GC.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

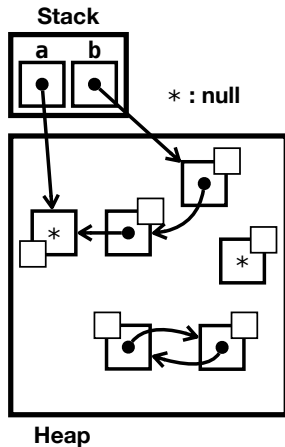
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Assume that the GC is triggered at the following program point, and let's perform **mark-and-sweep** GC.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

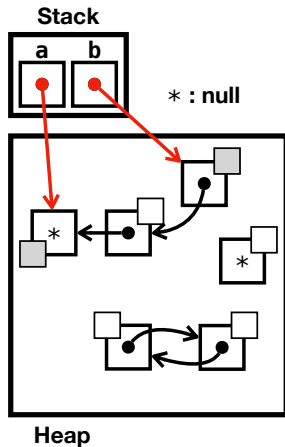
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Assume that the GC is triggered at the following program point, and let's perform **mark-and-sweep** GC.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

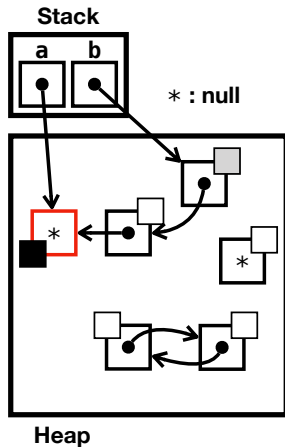
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Assume that the GC is triggered at the following program point, and let's perform **mark-and-sweep** GC.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

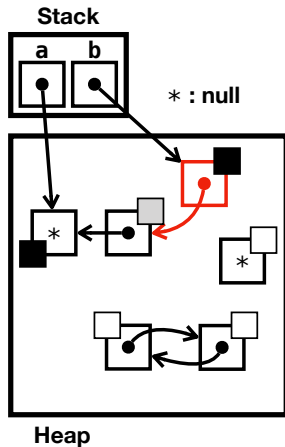
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Assume that the GC is triggered at the following program point, and let's perform **mark-and-sweep** GC.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

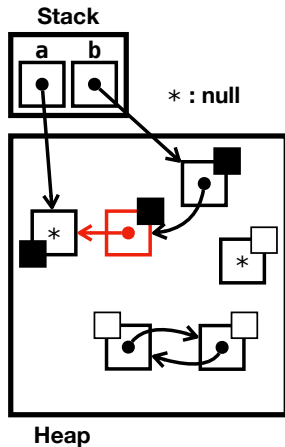
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Assume that the GC is triggered at the following program point, and let's perform **mark-and-sweep** GC.

```
case class Box(var x: Box)

var a = Box(Box(Box(null)))

var b = Box(Box(null))

a.x.x.x = a.x

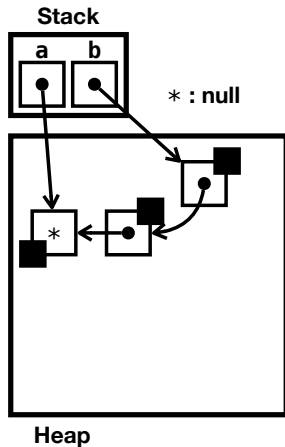
b.x = a

a.x = Box(null)

a = a.x

b = null
```

*



Mark-and-Sweep GC is one of the **tracing GC** algorithms that **traverses the heap** to find unreachable objects when it is **triggered** under some conditions.

Pros:

- **Handle Reference Cycles** – It can deallocate unreachable reference cycles.

Cons:

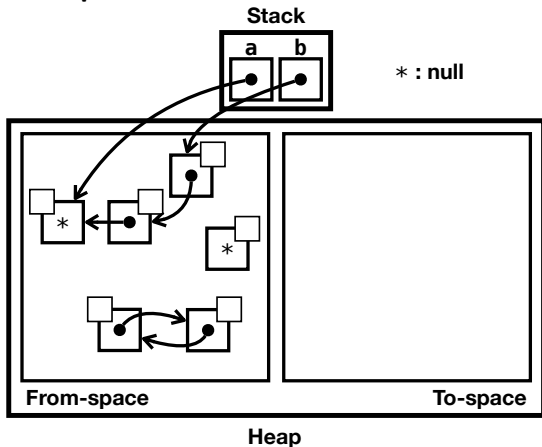
- **Stop-the-world** – It **stops** the program execution during GC.
- **Free List** and **Fragmentation** – It requires a **free list** to keep track of available free memory cells in heap, and it also suffers from **fragmentation** making it difficult to allocate large objects.

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it splits the heap into two spaces, **from-space** and **to-space**, and **copies** all the reachable objects into a **compact** layout:

- **Allocation** – It allocates memory cells only in **from-space**.
- **Deallocation** – It deallocates all the unreachable objects as follows:
 - ① **Mark** all memory cells as **unreachable** (white).
 - ② **Copy** all memory cells referenced by roots to the **to-space**, leave a **forwarding pointer** in each original cell to its copy, mark the copies as **unscanned** (gray), and **update** the roots to point to the copies.
 - ③ Repeat until there are no unscanned (gray) memory cells:
 - ① **Pick** an unscanned (gray) memory cell in the **to-space**.
 - ② **Copy** each memory cell referenced by the picked one to the **to-space** in the same way (gray), unless it already has a forwarding pointer.
 - ③ **Update** the references of the picked memory cell to point to the copies.
 - ④ **Mark** the picked memory cell as **scanned** (black).
 - ④ **Swap** from-space and to-space. All memory cells remaining in the old from-space are **deallocated**.

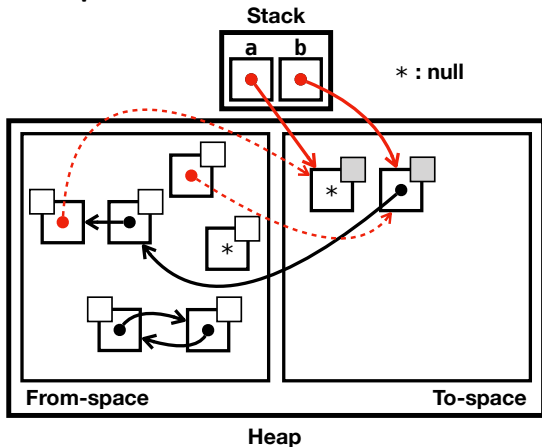
Copying GC (Two-Space GC)

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.



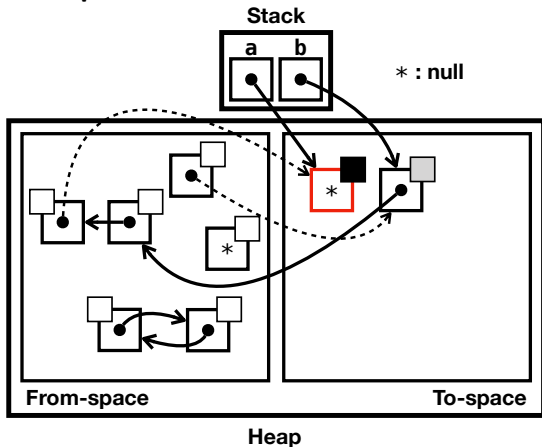
Copying GC (Two-Space GC)

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.



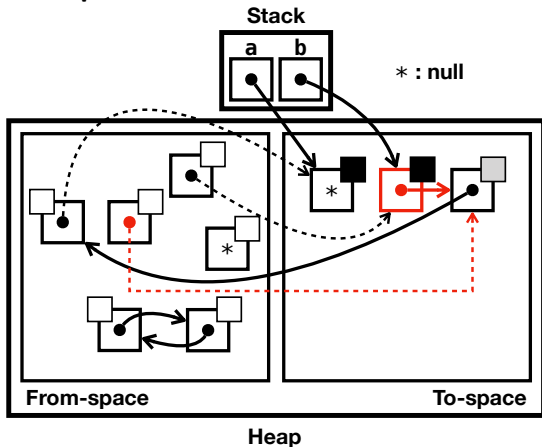
Copying GC (Two-Space GC)

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.



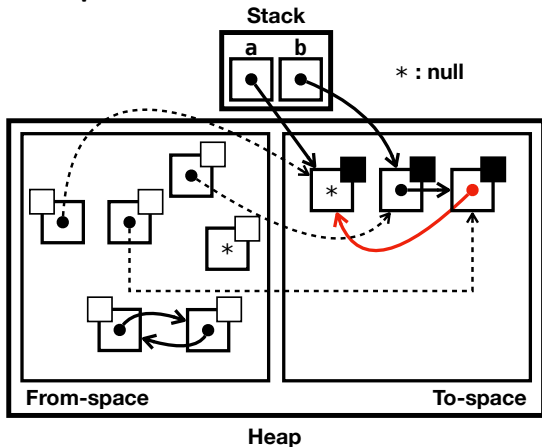
Copying GC (Two-Space GC)

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.



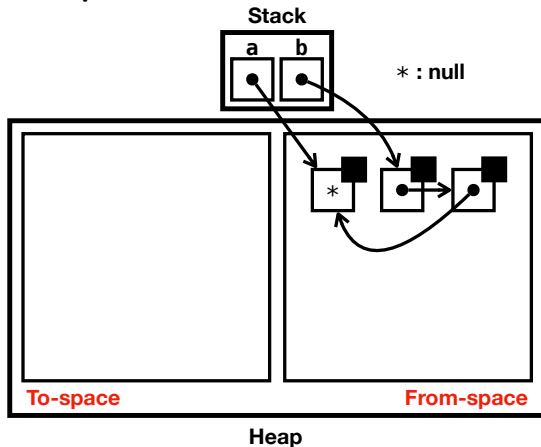
Copying GC (Two-Space GC)

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.



Copying GC (Two-Space GC)

Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.



Similar to mark-and-sweep GC, **copying GC** (Two-space GC) is another **tracing GC** algorithm. However, it **copies** all the reachable objects and reorganizes them in a **compact** layout by splitting the heap into two spaces: **from-space** and **to-space**.

Pros:

- **Handle Reference Cycles** – It can deallocate unreachable reference cycles.
- **No more Free List and Fragmentation** – The heap is always **contiguous** after GC.
- **Fast Allocation** – It only needs to track the first free cell.

Cons:

- **Stop-the-world** – It **stops** the program execution during GC.
- **Only half of the heap** (from-space) is used for allocation.
- **Expensive copying** – It copies all reachable objects on every GC.

Real-world programming languages use more sophisticated GC algorithms, combine them, or let users choose among them:

- **Generational GC** – Collects **young** objects more often: e.g., Java¹, Python (for reference cycles)²
- **Concurrent GC** – Runs GC **concurrently** with the program to reduce pauses: e.g., Java³, Golang⁴

They also use **compiler optimizations** to reduce heap allocations:

- **Escape Analysis** – Allocates an object on the **stack** if it does not **escape** the function: e.g., Java⁵, Golang

Or, a totally different approach, the **ownership** system, is used in Rust.⁶

¹<https://www.oracle.com/webfolder/technetwork/tutorials/obe/java/gc01/index.html>

²https://github.com/python/cpython/blob/main/InternalDocs/garbage_collector.md

³<https://docs.oracle.com/en/java/javase/26/gctuning/z-garbage-collector1.html>

⁴<https://tip.golang.org/doc/gc-guide>

⁵<https://docs.oracle.com/en/java/javase/26/vm/java-hotspot-virtual-machine-performance-enhancements.html>

⁶<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>

- **Stack and Heap** – The stack frame of a function is deleted when the program exits the function, but the heap needs other techniques.
 - **Tail-call optimization (TCO)** discards the current stack frame before a tail call.
- **Manual Memory Management** – Efficient, but error-prone (memory leak, use after free, and double free).
- **Garbage Collection (GC)** – Automatically deallocates unreachable memory cells in heap.
 - **Reference counting** – No pauses, but cannot handle reference cycles.
 - **Mark-and-sweep GC** – Handles reference cycles, but stop-the-world and fragmentation.
 - **Copying GC** – Compact heap and fast allocation, but uses only half of the heap.
 - **Others** – Generational GC, concurrent GC, escape analysis, and ownership.

- Lazy Evaluation

Jihyeok Park

jihyeok_park@korea.ac.kr

<https://plrg.korea.ac.kr>